



Study Material

(Object Oriented Programming in JAVA - BCA40107)

Table of Contents

Module 1: Threads & Applet

- I. Fundamentals of Java Multithreading
 - II. Basics of Applet Programming
 - III. MCQs, Short Questions, Long Questions
-

I. Fundamentals of Java Multithreading

Multithreading is a specialized form of multitasking that allows a single program to perform two or more tasks concurrently. Java's multithreading model is uniquely efficient because it is **thread-based** rather than process-based. In process-based multitasking, the program is the smallest unit of code, and each process requires its own address space, leading to significant overhead. In contrast, Java threads share the same address space and cooperatively share the same process, making inter-thread communication inexpensive and context switching low-cost.

1. The Main Thread

Every Java program has at least one thread of execution called the **main thread**. It starts running immediately upon program execution.

- **Role:** It is the thread from which all "child" threads are spawned. It must often be the last thread to finish because it performs various shutdown actions, such as closing files or network connections.
- **Control:** You can control the main thread by obtaining a reference to it via the `Thread.currentThread()` method. This allows you to set its name (via `setName()`), its priority, or force it to pause (via `sleep()`).

2. The Thread Life Cycle

A thread exists in a specific state at any given moment. Understanding these transitions is key to managing concurrency:

- **NEW:** A thread that has been instantiated but not yet started.
- **RUNNABLE:** A thread that is either currently executing or is ready to run as soon as it gains access to the CPU.

- **BLOCKED:** A thread that is waiting for a monitor lock to enter a synchronized block or method.
- **WAITING:** A thread that is suspended indefinitely while waiting for another thread to perform a specific action (e.g., calling `notify()`).
- **TIMED_WAITING:** A thread that is suspended for a specific period (e.g., during `Thread.sleep()` or a timed `wait()`).
- **TERMINATED:** A thread that has completed its execution and cannot be restarted.

3. Creation of Multiple Threads

Java provides two primary mechanisms for spawning new threads:

- **Implementing the Runnable Interface:** You define a class that implements the `run()` method. This is the preferred method if your class needs to inherit from a different superclass, as Java does not support multiple inheritance of classes.
- **Extending the Thread Class:** You create a subclass of `Thread` and override its `run()` method. This is simpler for very basic threading tasks but limits the class's flexibility for future inheritance.

4. Thread Priorities

Java uses an integer-based priority system (ranging from 1 to 10) to help the scheduler decide when to switch contexts.

- **Constants:** `MIN_PRIORITY` (1), `NORM_PRIORITY` (5), and `MAX_PRIORITY` (10).
- **Preemptive Multitasking:** High-priority threads can **preempt** lower-priority ones. If a high-priority thread wants to run, it usually does so immediately, suspending any lower-priority threads currently using the CPU.

5. Thread Synchronization and Monitors

To prevent **race conditions**—where multiple threads attempt to modify shared data simultaneously—Java uses **synchronization**.

- **Monitors:** Every object in Java has an implicit monitor (a mutually exclusive lock). Only one thread can own a monitor at a time.
- **Synchronized Methods:** By adding the `synchronized` keyword to a method, you ensure that only one thread can enter that method for a specific object instance.
- **Synchronized Statements:** You can synchronize a specific block of code by referencing a target object's monitor: `synchronized(object) { // code }`.

6. Inter-thread Communication

Java avoids the inefficiency of "polling" (looping to check a condition) through three methods defined in the `Object` class:

- **wait():** Tells the calling thread to give up the monitor and sleep until another thread calls `notify()`.
- **notify():** Wakes up a single thread that is waiting on the object's monitor.
- **notifyAll():** Wakes up all threads waiting on that monitor; the scheduler then decides which one runs.

7. Deadlock

Deadlock is a runtime error where two threads have a circular dependency on a pair of synchronized objects. Thread A holds a lock on Object X and waits for Object Y, while

Thread B holds a lock on Object Y and waits for Object X. Neither thread can proceed, and the program hangs indefinitely.

8. Suspending and Resuming Threads

Early Java versions used `suspend()`, `resume()`, and `stop()`, but these are now **deprecated**.

- **Danger:** `suspend()` could cause deadlocks if a thread was paused while holding a critical lock. `stop()` could leave data in a corrupted state.
- **Modern Solution:** Use a **flag variable** (checked within a loop in the `run()` method) that signals the thread to pause, resume, or terminate itself safely.

II. Introduction to Applets

Applets are small applications designed to be **transmitted over the Internet** and executed by a Java-compatible web browser or an **applet viewer**. There are two primary varieties: those based directly on the **Applet class**, which use the **Abstract Window Toolkit (AWT)**, and those based on the **JApplet class**, which use **Swing** to provide a richer user interface. Unlike standard Java applications, applet execution **does not begin at main()**; instead, it is started and controlled by a different mechanism.

I. Applet Architecture

Applets are inherently **event-driven**, functioning like a set of interrupt service routines. An applet typically waits for a user to trigger an interaction, such as a **mouse click** or a **keystroke**, which is sent to the applet as an **event** that it must respond to. Critically, an applet must perform its task and then **quickly return control** to the run-time system to ensure the user interface remains responsive. To perform repetitive tasks, such as displaying a scrolling message, an applet must start its own **additional thread of execution**.

II. The Applet Life Cycle

AWT-based applets follow a strict sequence of method calls that provide a basic mechanism for the browser or applet viewer to interface with the code. These methods are:

- **init():** This is the **first method** called and is used to initialize variables or perform one-time setup tasks.
- **start():** Called after `init()`, this method is executed whenever the applet is **started or restarted**. It runs every time a user returns to the HTML document containing the applet.
- **paint(Graphics g):** This method is called whenever the **applet's output must be redrawn**, such as when its window is uncovered or minimized and then restored.
- **stop():** This is called when the browser leaves the HTML document, allowing the applet to **suspend threads** that do not need to run when it is not visible.
- **destroy():** This final method is called when the environment determines the applet must be **removed from memory** to perform shutdown activities.

III. Simple Display and Interaction

Output within an AWT-based applet window is handled through the **Graphics class**, specifically the **drawString()** method. This method displays a string at a specified X,Y coordinate, where **0,0 is the upper-left corner** of the window. Applets can also set their window colors using **setBackground()** and **setForeground()**. When the applet needs to

update its window information, it calls **repaint()**, which requests that the AWT run-time system execute a call to the applet's **paint()** method.

IV. Deployment and Parameters

Applets can be specified directly in an **HTML file** using the **APPLET tag**, which indicates the width, height, and the compiled **.class** file to be loaded. This tag also allows **parameters** to be passed to the applet using the **PARAM tag**. The applet retrieves these values during execution using the **getParameter()** method, which returns the value of the parameter as a string. Additionally, an applet can output messages to the browser's **status window** by calling **showStatus()**, which is useful for feedback or debugging.

V. Security and Portability

To prevent security breaches, applets run within a "**sandbox**," which limits their access to local resources such as the file system. This ensures that downloading an applet does not risk virus infection or unauthorized access to data. Portability is achieved through **Java bytecode**, which allows the same applet code to work across the diverse universe of computers and operating systems connected to the Internet.

Analogy for Applet Life Cycle: Think of an applet like a **guest performer at a theater**.

- **init()** is the performer arriving at the theater and setting up their equipment.
- **start()** is the curtain rising and the performer beginning the show.
- **paint()** is the performer adjusting their position or performance every time the audience moves or the lighting changes.
- **stop()** is the performer taking a break and going backstage when the audience leaves for intermission.
- **destroy()** is the performer packing up their gear and leaving the theater for good once the engagement is over.

Summary Visual Guide (Text-Based Infographic)

Feature	Multithreading	Applets
Smallest Unit	Thread (Dispatchable code)	Component (Visual element)
Control Flow	Sequential or Parallel	Event-Driven
Standard Setup	Thread Or Runnable	init, start, paint, stop, destroy
Safety	Synchronization / Monitors	Sandbox (Restricted access)
Key Interaction	wait(), notify(), join()	Listeners (Source/Listener model)

Analogy for Multithreading Synchronization: Think of a **single-user office printer** as a **shared resource**. The **monitor** is the small "**In Use**" **light** on the printer. If three employees (threads) want to print a document, only the one who currently has the light on (has acquired the monitor) can print. The others must wait in line (the blocked state). If the printer runs out of



paper, the current employee might **wait()** by the printer, letting others check if they have paper. Once someone refills the tray and presses a button (**notify()**), the first employee can resume their task. **Deadlock** would occur if Employee A is waiting for a key from Employee B to open the printer, but Employee B is waiting for Employee A to provide the ink cartridge before giving up the key—neither can move, and both wait forever.

Practice Questions (MCQ, Short, Long) with Answers

Part A: Multiple Choice Questions (MCQs)

1. In a thread-based multitasking environment, what is the smallest unit of dispatchable code?
A) Process
B) Program
C) Thread
D) Buffer

2. Which static method in the Thread class is used to obtain a reference to the thread in which it is called?
A) main()
B) run()
C) start()
D) currentThread()

3. What is the range for thread priorities defined in the Thread class?
A) 0 to 1
B) 1 to 10
C) 1 to 5
D) 0 to 100

4. Which class defines the wait(), notify(), and notifyAll() methods used for inter-thread communication?
A) Thread
B) Runnable
C) Object
D) System

5. Which of the following is the correct sequence of method calls when an AWT-based applet first begins execution?
A) start(), init(), paint()
B) init(), paint(), start()



- C) init(), start(), paint()
- D) paint(), init(), start()

6. To begin the execution of a new thread, which method must be called to invoke the run() method?

- A) run()
- B) start()
- C) execute()
- D) init()

7. Why were the suspend(), resume(), and stop() methods in the Thread class deprecated?

- A) They were too slow for modern processors.
- B) They could cause serious system failures, such as deadlocks or corrupted data.
- C) They were only applicable to applet-based threads.
- D) They did not support the Runnable interface.

8. In the Delegation Event Model, what is the term for an object that generates an event when its internal state changes?

- A) Source
- B) Listener
- C) Delegate
- D) Monitor

9. Which interface must a listener implement to handle the event generated when an AWT Button is pressed?

- A) ItemListener
- B) MouseListener
- C) ActionListener
- D) WindowListener

10. Which thread state indicates that a thread has suspended execution because it is waiting to acquire a monitor lock?

- A) WAITING
- B) TIMED_WAITING
- C) BLOCKED
- D) NEW



Part B: Short Answer Questions

1. What is the importance of the main thread in a Java application, and why is it usually the last thread to finish execution?
2. Which two distinct mechanisms does Java provide for a programmer to create a new thread of execution, and what is the primary difference between them?
3. Under what specific conditions will a context switch occur between threads, and how does preemptive multitasking affect this process?
4. How does a monitor function as a mutually exclusive lock to prevent race conditions when multiple threads access a shared resource?
5. Which specific methods defined in the Object class are used to implement inter-thread communication to avoid the efficiency loss of polling?
6. What specific circular dependency condition leads to a deadlock between two or more threads and synchronized objects?
7. For what technical reasons were the suspend(), resume(), and stop() methods in the Thread class deprecated, and what is the modern alternative for controlling thread execution?
8. What is the standard sequence of method calls that occurs when an AWT-based applet first begins its execution, and what is the purpose of each?
9. In the delegation event model, how is an event source defined, and how does it interact with an event listener?
10. Which AWT class is used to create a single-line text-entry area for user interaction, and what event is generated when a user presses enter within it?



Part C: Long Answer Questions

1. Compare and contrast process-based multitasking with thread-based multitasking. Explain how Java's multithreading model utilizes a shared address space and the concept of "lightweight" execution to improve program efficiency over the traditional "heavyweight" process model.
2. Detail the two primary mechanisms for creating a thread in Java: implementing the Runnable interface and extending the Thread class. In what scenarios is implementing Runnable considered a superior design choice, particularly regarding the constraints of class inheritance and the flexibility of the class hierarchy?
3. Discuss the unique role and significance of the "main thread" in a Java application. Beyond being the entry point of execution, how does it interact with child threads, and why is it generally recommended that the main thread be the last to finish execution?
4. Elaborate on the necessity of thread synchronization in concurrent programming. Define the concept of an object's monitor as a mutually exclusive lock and explain how the synchronized keyword prevents race conditions and data corruption when multiple threads access shared resources.
5. Explain the mechanics of inter-thread communication in Java. Specifically, discuss how the wait(), notify(), and notifyAll() methods, defined in the Object class, allow threads to communicate their state to one another to eliminate the need for inefficient polling.
6. Define the technical conditions that lead to a "deadlock" in multithreaded applications. Provide an example of how circular dependencies between two or more synchronized objects can cause a program to hang indefinitely and explain why this specific runtime error is difficult to debug.
7. Analyze the lifecycle of an AWT-based applet by detailing its five primary methods: init(), start(), paint(), stop(), and destroy(). How do these methods coordinate with the browser environment to manage the applet's execution when a user navigates between web pages?
8. Describe the various states in a thread's lifecycle, including NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. What specific actions or conditions cause a thread to transition between these states, and why is the BLOCKED state critical to synchronization?
9. Discuss the Delegation Event Model used for handling user interaction in Java GUIs. Explain the roles of event sources and event listeners, and describe the process of registration that allows a component, such as a button, to delegate event processing to a specific handler.



10. Analyze the reasoning behind the deprecation of the `suspend()`, `resume()`, and `stop()` methods in the `Thread` class. What are the potential system failures and stability risks associated with these methods, and how does the modern approach using flag variables provide a safer alternative?

End of Module 5